

to add data in one memory location to data in another memory location. A RISC processor on the other hand would require multiple steps, first instructing the CPU to load data from both memory locations onto registers one by one, and then instruction to add them, and finally an instruction to place this data back in memory.

It might seem like CISC is the better bet here, as it seems faster doesn't it? But the fact is, the CISC CPU still has to do all the things the RISC CPU would do. The fact that it exposes them as a single instruction doesn't change the amount of work behind that instruction. The CISC add operation might end up taking as much time, or even more than the equivalent RISC instructions.

The most famous examples of a CISC CPUs are those by Intel and AMD, x86 in other words. RISC is well represented by ARM, MIPS, AVR, PowerPC and most other CPU designs.

It is more an accident of history that most desktop and laptop computers use Intel / AMD CISC CPUs while most mobile platforms use ARM or MIPS RISC CPUs. In fact a number of TOP500 supercomputers in the world are of RISC design. The PlayStation 3s Cell processor too was a RISC processor.

Over time as the x86 RISC processors have advanced, there has been an increase in the number of instructions, and their capabilities. Commonly performed sequences of instructions have been converted into optimised single instructions. Think of buzz words like MMX, 3DNow!, SSE, SSE2 etc. They are all extensions to the instruction set of x86 CPUs. Many of these are powerful instructions that can efficiently perform repeated operations on larger data sets.

Knowledge of assembly also means you can take advantage new developments before compiler designers have taken them into account, and optimised code output to use them. In fact some special CPU instructions are only available in Assembly!

A programmer well versed in assembly is also one with good knowledge of the underlying hardware, and this pays off even when using higher level languages. The way hardware is designed means that often minor changes in code (such as switching the inner and outer loops in a nested loop) can have a huge impact on the speed. Why? How? When? These are the question that can only be answered by learning more about machine architectures.

Despite the proliferation of higher-level languages that hide all the complexity we just talked about, there is still great value in Assembly language, and knowing it can be very beneficial even if you don't ever need to code in it.